```
% MAE263C-Jumping Leg Simulation
% Author: Conrad Ku (Adapted from the HOPPY Simulator)
% Last modified: 06/11/2025

clear all
close all
clc

addpath generate_functions\
addpath controllers\
addpath dynamics\
addpath generate_functions\
addpath events\
addpath plotting\

% --- parameters ---
% Get the parameters about the robot and simulation settings
p = get_params();
params = p.params;
N_jumps = 3;
dt = 0.01;
contact_state = zeros(1);
Q_des = fcn_IK(p.foot_d,params)';
p_des = p.foot_d';

%Initial condition:
q0 = [0.5; -pi/4; pi/2];
dq0 = [0; 0; 0];
ic = [q0; dq0;];

% Checks if the Foot is already on the Ground From ICs
foot_init = fcn_p4(q0,params);
if foot_init(2)<0
   fprintf('Start Above the Floor! \n')
   return
end


% Time Lengths
t_start = 0;
t_jump = 2; % Time Per Jump
t_final = t_jump * (N_jumps+1);
% Data Logging Variable Initialization
t_out = t_start; % Simulation Time
X_out = ic'; % States
U_out = [ 0 0 ]; % Input Torques
F_out = [ 0 0 ]; % GRFs
P_out = fcn_p14(q0,params)';


% Debugging Jump Controller
test.x = zeros(0); % debugging phase variable
test.F = zeros(3,0);
test.tau = zeros(3,0);

p.time_contact = zeros(2,0); % Stores contact time in (1), then stance time in (2)

%% Starting Dynamics Solver
fprintf('Aerial Phase \n')
% Iterate Through # of Hops
for i=1:N_jumps+1
   if t_start == t_final
      fprintf('Simulation Ended \n')
      break
   end
   %Simulate unconstrained dynamics in the Aerial Phase
   options = odeset('Events',@(t,X)event_contact(t,X,p));
   [t,X] = ode45(@(t,X)robot_dyn_aerial(t,X,p),[t_start:dt:t_final, t_final], X_out(end,:),options); %, t_final
   % Stops upon Contact, outputs the time and states
   %-------------------------------------------------------------------------
   p.time_contact(:,end+1) = [t(end);0] ;
   p.pt_contact = fcn_p4(X(end,1:3),params);
   % Store Aerial Phase Variables
   n_timesteps = length(t);
   t_out = [t_out; t(2:n_timesteps)];
```

```
X_out = [X_out ;X(2:n_timesteps,:)];
[~,U,F] = robot_dyn_aerial(t,X,p); % Collect Input Torques and Ground Forces
U_out = [U_out ; U(2:n_timesteps,:)];
F_out = [F_out ; F(2:n_timesteps,:)];
t_start = t_out(end); % Log the new Start Time for the Next Phase
contact_state = [contact_state; zeros(length(t(2:n_timesteps)),1)];
Q_des = [Q_des; repmat(fcn_IK(p.foot_d,params)',length(t(2:n_timesteps)),1)];
p_des = [p_des;repmat(p.foot_d',length(t(2:n_timesteps)),1)];
% Operational Space Control in the Air
p_out = zeros(length(t),2);
for j=1:size(X,1)
   p_out(j,:) = fcn_p14(X(j,1:3),params)';
end
P_out = [P_out; p_out(2:n_timesteps,:)];
% Plot
figure()
plot(t,p_out(:,1),'r','LineWidth',2)
hold on
plot(t,p_out(:,2),'g','LineWidth',2)
yline(p.foot_d(1),'--r','LineWidth',2)
yline(p.foot_d(2),'--g','LineWidth',2)
legend('X','Y')

% Calculate the inelastic Impact Event
X_minus = X_out(end,:);
X_plus  = fcn_impact(X_minus,p);
X_out(end,:) = X_plus'; %Rewrite value to Include Impact Force
fprintf('Landing at %.3f s:\n',t(end))
if i == N_jumps+1 % Have it just Stand and Hold Pose after Final Jump
   %Simulate Standing and Pose Maintenance
   if t_start == t_final
      fprintf('Simulation Ended \n')
      break
   end
   [t,X] = ode45(@(t,X)robot_dyn_stance_stand(t,X,p),[t_start:dt:t_final,t_final],X_out(end,:));
   p.time_contact(2,end) = t(end);
   % Store Stance Phase Variables
   n_timesteps = length(t);
   t_out = [t_out; t(2:n_timesteps)];
   X_out = [X_out ;X(2:n_timesteps,:)];
   [~,U,F] = robot_dyn_stance_stand(t,X,p); % Collect Input Torques and Ground Forces
   U_out = [U_out ; U(2:n_timesteps,:)];
   F_out = [F_out ; F(2:n_timesteps,:)];
   contact_state = [contact_state; ones(length(t(2:n_timesteps)),1)];
   Q_des = [Q_des; repmat(fcn_IK(p.foot_d_stance,params)',length(t(2:n_timesteps)),1)];
   p_des = [p_des;repmat(p.foot_d_stance',length(t(2:n_timesteps)),1)];
   for j=1:size(X,1)
      p_out(j,:) = fcn_p14(X(j,1:3),params)';
   end
   P_out = [P_out; p_out(2:n_timesteps,:)];
   fprintf('Holding Pose! \n')
   fprintf('Simulation Ended at %.3f s\n', t(end))
else % Jump
   if t_start == t_final
      fprintf('Simulation Ended \n')
      break
   end
   %Simulate constrained dynamics in the Stance Phase
   options = odeset('Events',@(t,X)event_flight(t,X,p));
   [t,X] = ode45(@(t,X)robot_dyn_stance_jump(t,X,p,test),[t_start:dt:t_final,t_final],X_out(end,:),options); % , t_final
   % p.time_LO = t(end); % Re-evaluate need/purpose
   p.time_contact(2,end) = t(end);

   % Store Stance Phase Variables
   n_timesteps = length(t);
   t_out = [t_out; t(2:n_timesteps)];
   X_out = [X_out ;X(2:n_timesteps,:)];
   [~,U,F,test] = robot_dyn_stance_jump(t,X,p,test); % Collect Input Torques and Ground Forces
   U_out = [U_out ; U(2:n_timesteps,:)];
   F_out = [F_out ; F(2:n_timesteps,:)];
   contact_state = [contact_state; ones(length(t(2:n_timesteps)),1)];
   Q_des = [Q_des; repmat(fcn_IK(p.foot_d_stance,params)',length(t(2:n_timesteps)),1)];
   p_des = [p_des;repmat(p.foot_d_stance',length(t(2:n_timesteps)),1)];
   for j=1:size(X,1)
      p_out(j,:) = fcn_p14(X(j,1:3),params)';
```

```
    end
    P_out = [P_out; p_out(2:n_timesteps,:)];
    t_start = t_out(end); % Log the new Start Time for the Next Phase
    if t(end)==t_final
      fprintf('Could not Achieve Jump \n')
    else
      fprintf('%d of %d Jumps Achieved! \n',i,N_jumps)
    end
  end

end

% Check if Joint Limits Violated
if any(X(:,2) > p.hip_upper) || any(X(:,2) < p.hip_lower)
  fprintf("Hip Joint Violates Joint Limits! \n")
end
if any(X(:,3) > p.knee_upper) || any(X(:,3) < p.knee_lower)
  fprintf("Knee Joint Violates Joint Limits! \n")
end
% Check for Torque Limit Violations
% Hip
for i=1:length(t_out)
  omega = X_out(i,5);
  tau = U_out(i,1);
  torque_constraints = p.A_motor * [omega;tau] - p.B_motor;
  if any(torque_constraints>0)
    fprintf('Hip Motor Torque Exceeds Limits at Time: %f s \n', t_out(i))
  end
end
% Knee
for i=1:length(t_out)
  omega = X_out(i,6);
  tau = U_out(i,2);
  torque_constraints = p.A_motor * [omega;tau] - p.B_motor;
  if any(torque_constraints>0)
    fprintf('Knee Motor Torque Exceeds Limits at Time: %f s \n', t_out(i))
  end
end
% Base Acceleration
d_v = diff(X_out(:,4));
d_t = diff(t_out);
hip_accel = [ 0; d_v./d_t];
hip_force = [hip_accel*params(1)*(params(5)+params(6)+params(7))];

figure()
plot(t_out,X_out(:,1))
hold on
pos_d = p.foot_d_stance;
stance_time = [t_start:0.25:t_final, t_final];
plot(stance_time,-pos_d(2)*ones(length(stance_time)),'--k','LineWidth',2)
title(['Hip Height Over ',num2str(N_jumps), ' Jumps'])


%Animating the robot:
% animateRobot(t_out,X_out,p,0)

function p = get_params()
% Place to Pre-Load Global Static Variables, as well as pass them in and
% out of functions

%% Tuneable Gains
% Aerial Phase Foot Position
p.foot_d = [0 -0.2625]';
% Stance Phase Foot Position
p.foot_d_stance = [-0.005 -0.275]';

% Gains Matrices
%     Stance Phase
p.Kp_stance = diag(150 * [1 1.25]);
p.Kd_stance = diag(15* [1 1]);
%     Jump  Phase ( Joint Space )
p.Kp_jump = diag(1.5 * [1 0.75]);
p.Kd_jump = diag(0.0875 * [1 1]);

%     Aerial Phase ( Task Space X/Y)
```

```
p.Kp_aerial = diag(300 * [1.5 2]);
p.Kd_aerial = diag(75 * [0.625 0.875]);

% Jump Variables (Speedier Jump is better for liftoff - otherwise need more
% Force Commanded for a static Jump
p.time_stance = 1;
jump_magnitude = 15; % X times the static weight
% Peak Doesn't Usually Matter Since the Leg Lifts Off before the Full
% Trajectory - however, a higher peak results in a faster impulse


% Baumgarte Numerical Stabilization Values
p.alpha = 10;
p.beta = 10;
p.alpha_standing = 50;
p.beta_standing = 500;

% Liftoff Force Threshold
liftoff_percentage = 0.025;


%% parameters for matrix calculations

% g Gravity constant
% l1,l2,l3 Length of link 1-3
% M1,M2,M3 Mass of link 1-3
% J1,J2,J3 Moment of Inertia of Link 1-3

g = 9.81; %m/s^2

l1 = 0; % unit: meter
l2 = 0.125;
l3 = 0.215;

l2_COM_X = 0.01415979;
l2_COM_Y = 0.05919690;
l3_COM_X = 0.117055136 ;
l3_COM_Y = -0.019215548 ;


M1 = 0.67514650 ; % unit: kg
M2 = 0.48206937 ;
M3 = 0.06067653;
p.weight = (M1 + M2 + M3) * g;
p.jump_force = jump_magnitude * p.weight;
p.threshold = liftoff_percentage * p.weight;

p.torque_limit = 1.16*20;

% Hip Joint and Velocity Limits
p.hip_lower=-2.8;
p.hip_upper=0.46 ;
p.hip_velocity=31.67;
% Knee Joint and Velocity Limits
p.knee_lower=-2.34;
p.knee_upper=2.62;
p.knee_velocity=31.67;


% From Solidworks, taken at COM and aligned with Link Origin
% ----------------------------------------
%Links inertia tensors (J_ij = J_ji)
% Hip Link
%I1 = [[J1_xx J1_xy J1_xz];
%     [J1_xy J1_yy J1_yz];
%     [J1_xz J1_yz J1_zz];]; kg*m^2

J1_xx = 0.01;
J1_yy = 0.01;
J1_zz = 0.01;
J1_xy = 0.01;
J1_xz = 0.01;
J1_yz = 0.01;

% Femur Link
```

```
J2_xx =  0.00062745;
J2_yy = 0.00085866;
J2_zz = 0.00118754;
J2_xy = -0.00024930;
J2_xz = 0.00011797;
J2_yz = -0.00012745;

% Tibia Link
J3_xx = 0.00002438;
J3_yy = 0.00044109;
J3_zz = 0.00046370;
J3_xy = 0.00003292;
J3_xz = -0.00000008;
J3_yz =  0.00000001; % <<  Showed up as zero but set to non-zero to preserve stability

%Friction model Coulomb + viscous
tau1_Coulomb = 0;
tau2_Coulomb = 0;
tau3_Coulomb = 0;
b1_viscous = 0;
b2_viscous = 0;
b3_viscous = 0;

%Actuators reflected inertia matrix
N_KB = 9;
N_KBMB = 20;       % KBMB
Ir = 1.82e-3/(N_KB^2);   % Reflected Inertia From KB then divided by KB ratio, assumed same rotor
p.M_rotor = diag([0,N_KBMB^2 * Ir, N_KBMB^2 * Ir]);
% Motor Parameters - At Output
kt = 1.16; % N-m/A
kv = 9; % RPM/V
kv = kv * ((2*pi)/60); % rad/s/V
R_armature = 0.6; % ohm
% ke =

% Motor Specs
torque_limit = 20; % N-m
max_voltage = 24; % V
max_current = 20;
no_load_speed = kv*max_voltage;
stall_torque = kt * max_current;

p.A_motor = [ 0 , 1 ;
        0 , -1;
        stall_torque/no_load_speed , 1 ;
        -stall_torque/no_load_speed , -1 ; ];
p.B_motor = [torque_limit ; torque_limit ; stall_torque ; stall_torque];



params = [g, l1, l2, l3, M1, M2, M3, J1_xx, J1_yy, J1_zz, J1_xy, J1_xz,...
   J1_yz, J2_xx, J2_yy, J2_zz, J2_xy, J2_xz, J2_yz, J3_xx, J3_yy, J3_zz,...
   J3_xy, J3_xz, J3_yz,l2_COM_X,l2_COM_Y,l3_COM_X,l3_COM_Y];

p.friction = [tau1_Coulomb; tau2_Coulomb; tau3_Coulomb; b1_viscous; b2_viscous; b3_viscous];
p.params = params;
p.N_animate = 30;   % for animation time smoothness

function animateRobot(tin,Xin,p,record)

if size(Xin,2) == 6
   Xin = Xin';
end

%--------------------------------------------------------------------------
%ANIMATION
dtA = 0.05;          %Time step of animation
timeA = 0:dtA:tin(end);   %Time vector for animation with constant stepping

% record = 0;          %Set to 1 if would like to record video and 0 if not
if (record)
   v = VideoWriter('CRS_Robot.avi');
   open(v)
end
f = figure;
```

```
set(f, 'doublebuffer', 'on');
for k = 1:length(timeA)
  clf
  %Current robot state interpolation
  theta1 = interp1(tin,Xin(1,:),timeA(k));
  theta2 = interp1(tin,Xin(2,:),timeA(k));
  theta3 = interp1(tin,Xin(3,:),timeA(k));
  plotRobot([theta1; theta2; theta3],p);

  hold off
  F = getframe(f);
  drawnow;

  if(record)
    writeVideo(v,F)
  end
end
%Closing file
if(record)
  fprintf("Recording Saved to CRS_Robot.avi!")
  close(v)
end

end

function plotRobot(X,p)

params = p.params;

q = X(1:3);
% p1 = [0 0]';
p2 = fcn_p2(q,params);
p3 = fcn_p3(q,params);
p4 = fcn_p4(q,params);

chain = [p2 p3 p4];

rectangle('Position',[p2(1)-params(3)/4,p2(2)-params(3)/4, params(3)/2,params(3)/2],'FaceColor','blue', 'EdgeColor','none')
hold on
plot(chain(1,:),chain(2,:),'color','black','linewidth',4);
plot(chain(1,:),chain(2,:),'ocyan','LineWidth',4);

yline(0,'--black')
grid on
axis square

limits = (params(3)+params(4))*[-1.1 , 1.1 , -0.1, 1.1] + [ 0, 0, 0, q(1)];
% limits = [-0.05 0.05 -0.05 0.05];
axis(limits)
xlabel('x [m]','fontsize',12)
ylabel('y [m]','fontsize',12)


end

clc
close all
%Actuators reflected inertia matrix
N_KB = 9;
N_KBMB = 20;       % KBMB

%Actuators reflected inertia matrix
N_KB = 9;
N_KBMB = 20;       % KBMB

% Motor Parameters - At Output
kt = 1.16; % N-m/A
kv = 9; % RPM/V
kv = kv * 6;
% kv = kv * ((2*pi)/60); % rad/s/V
R_armature = 0.6; % ohm
% ke =

% Motor Specs
torque_limit = 25; % N-m
```

```
max_voltage = 24; % V
max_current = 20;
no_load_speed = kv*max_voltage; % deg/s
stall_torque = kt * max_current;

A_motor = [ 0 , 1 ;
        0 , -1;
        stall_torque/no_load_speed , 1 ;
        -stall_torque/no_load_speed , -1 ; ];
B_motor = [torque_limit ; torque_limit ; stall_torque ; stall_torque];

% Plotting Aided by ChatGPT then edited

% Generate torque-speed polygon using polytope
omega = linspace(-no_load_speed*2.08, no_load_speed*2.08, 1000); % rad/s
tau_bounds = zeros(2, length(omega)); % [lower; upper] torque bounds

for i = 1:length(omega)
  w = omega(i);

  % Solve A*v <= B for each omega slice (1D LP to find tau bounds)
  % A*v = A*[w; tau] = a1*w + a2*tau <= b  =>  linear constraints on tau
  % Extract all allowable tau at this omega
  a1 = A_motor(:,1);
  a2 = A_motor(:,2);
  b = B_motor - a1 * w;

  % Feasible tau region at this omega
  tau_upper = min(b(a2 > 0) ./ a2(a2 > 0));
  tau_lower = max(b(a2 < 0) ./ a2(a2 < 0));

  % Store
  tau_bounds(1,i) = tau_lower;
  tau_bounds(2,i) = tau_upper;
end

% Plot the filled polygon with transparency
figure;
hold on;



% Plot individual constraints (dotted, different colors)
colors = lines(4);
legend_entries = {};


plot(omega(1:477), torque_limit * ones(size(omega(1:477))), ':', 'LineWidth', 3, 'Color', colors(1,:));
legend_entries{end+1} = '\tau \leq limit';


plot(omega(520:end), -torque_limit * ones(size(omega(520:end))), ':', 'LineWidth', 3, 'Color', colors(2,:));
legend_entries{end+1} = '\tau \geq -limit';

slope = stall_torque / no_load_speed;
plot(omega(477:end), -slope * omega(477:end) + stall_torque, ':', 'LineWidth', 3, 'Color', colors(3,:));
legend_entries{end+1} = 'Upper Torque-Speed Limit';

plot(omega(1:520), -slope * omega(1:520) - stall_torque, ':', 'LineWidth', 3, 'Color', colors(4,:));
legend_entries{end+1} = 'Lower Torque-Speed Limit';

fill([omega fliplr(omega)], ...
   [tau_bounds(1,:) fliplr(tau_bounds(2,:))], ...
   [0.2 0.6 0.8], ...
   'EdgeColor', 'none', ...
   'FaceAlpha', 0.3);
xline(0,"--k",'LineWidth',2)
yline(0,"--k",'LineWidth',2)

% Labels and appearance
% xlim([-50,50])
xlabel('Speed \omega (deg/s)');
ylabel('Torque \tau (N·m)');
title('Motor Torque-Speed Limit Polygon');
legend(legend_entries, 'Location', 'best');
```

```
grid on;
hold off;
%%
[tau_min,tau_max] = torque_bounds(2000)
%%
function [tau_min, tau_max] = torque_bounds(omega)
  % Constants
  kv = 9 * 6;         % deg/s/V
  no_load_speed = kv * 24;    % max voltage = 24 V
  stall_torque = 1.16 * 20;   % kt = 1.16 Nm/A, max current = 20 A

  % Polygon constraints: A*[omega; tau] <= B
  A = [ 0 1; 0 -1;
       stall_torque/no_load_speed 1;
      -stall_torque/no_load_speed -1 ];
  B = [23.2; 23.2; stall_torque; stall_torque];

  b = B - A(:,1) * omega;

  % Compute torque bounds
  up = min(b(A(:,2)>0) ./ A(A(:,2)>0,2));
  lo = max(b(A(:,2)<0) ./ A(A(:,2)<0,2));

  if isempty(up) || isempty(lo) || lo > up
    [tau_min, tau_max] = deal(NaN);
  else
    tau_min = lo;
    tau_max = up;
  end
end

function [u,F] = controller_aerial(t,X,p)
q = X(1:3,:);
dq = X(4:6,:);
params = p.params;
% Ensure that Unobservable States do not affect Controller
observability = [ 0 0 0 ;
            0 1 0 ;
            0 0 1 ; ] ;
q = observability * q;
dq = observability * dq;
% Control Gains in the Air (Task Space)
Kp = p.Kp_aerial;
Kd = p.Kd_aerial;

% Desired Foot Position w.r.t Base Link
% Make Sure within Feasible Workspace, No checks implemented yet
% --------------------------------
foot_d = p.foot_d;

% Forward Kinematics w.r.t. Base Link
foot = fcn_p14(q,params); % p1_4
% Velocity of the foot relative to the base, ignores the rising/falling of
% the base in the control calculations
J14 = fcn_J14(q,params);
v_foot =  J14 * dq;

% Aerial Task Space PD Controller
% Change Control Law if Necessary
% --------------------------------
F = Kp * (foot_d-foot) - Kd * v_foot;
u = J14' * F;

function [u,test] = controller_jump(t,X,p,test)
params = p.params;
q = X(1:3);
dq = X(4:6);
time_stance = p.time_stance; % Stance Duration
if ~isfield(p,'time_contact')
  time_contact = 0; % If Starting from the Ground
else
  time_contact = p.time_contact(1,end); % Start Time of Stance Phase
end
% Ensure that Unobservable States do not affect Controller
observability = [ 0 0 0 ;
```

```
            0 1 0 ;
            0 0 1 ; ] ;
q = observability * q;
dq = observability * dq;
% Control Gains in the Air (Task Space)
Kp = p.Kp_jump;
Kd = p.Kd_jump;
% Phase Variable
% t
% p.time_contact
x = (t - time_contact)/time_stance;
% Ensure Phase is bounded between 0 and 1 for any Purpose
%\
x(x<0) = 0 ;
x(x>1) = 1 ;
test.x = [ test.x , x ];

% Feedforward Force Trajectory (6th Order)
% Only +Y force wanted, X forces unwanted due to Gantry
F_ff = zeros(2,1);
F_ff(2) = fcn_F_traj(x,0.875,0.125,p.jump_force);
% % Step Force
% if x>0.1
%     F_ff(2) = p.jump_force;
% end

% Feedback Torque using Weak PD Controller to Stabilize System About Set
% Point in Joint Space
x_des = p.foot_d_stance; % Desired Task Space Position of the Foot w.r.t Base
q_des = fcn_IK(x_des,params);
tau_fb = [0;Kp * (q_des - q(2:3)) - Kd * dq(2:3)];
J14 = fcn_J14(q,params);

% % Feedback Force Task Space
% x_des = p.foot_d_stance; % Desired Task Space Position of the Foot w.r.t Base
% x = fcn_p14(q,params);
% % Jacobian from Hip to Foot
% J14 = fcn_J14(q,params);
% x_dot = J14*dq;
% q_des = fcn_IK(x_des,params);
% f_fb = (Kp * (x_des - x) - Kd * x_dot);
% tau_fb = J14' * f_fb;
% Total Torque
test.F = [test.F, F_ff];

test.tau = [test.tau,  tau_fb ];

u = 1 *  (-(J14'*F_ff) + tau_fb); % '-' because it is Foot on Environment

function [u,test] = controller_stand(t,X,p,test)
params = p.params;
q = X(1:3);
dq = X(4:6);
G = fcn_Ge(q,params);

% Ensure that Unobservable States do not affect Controller
observability = [ 0 0 0 ;
            0 1 0 ;
            0 0 1 ; ] ;
q = observability * q;
dq = observability * dq;
% Control Gains in the Ground (Task Space)
Kp = p.Kp_stance;
Kd = p.Kd_stance;


% Feedback Torque using Controller to Stabilize System About Set Point in Task Space
x_des = p.foot_d_stance; % Desired Task Space Position of the Foot w.r.t Base
x = fcn_p14(q,params);
% Force Feedforward (Static Weight)
f_ff = [0;p.weight];
% Jacobian from Hip to Foot
J14 = fcn_J14(q,params);
x_dot = J14*dq;
% q_des = fcn_IK(x_des,params);
```

```
f_pd = Kp * (x_des - x) - Kd * x_dot;
u = (J14' * f_pd) -(J14' * f_ff) + G; % Gravity Compensation to Reduce SS Errors

function f = fcn_disturbance(t,p)%,X,p) % Applied Step Force to the Base
% params = p.params;
% q = X(1:3);
% J2t = fcn_J2(q,params)';
magnitude = 20;
start_time = 0.5;
step_duration = 1;


if (start_time<=t) && (t<start_time+step_duration)
   f = [0 , magnitude]';
elseif (2*start_time+step_duration<=t) && (t< 2*start_time + 2*step_duration)
   f = [0 , -magnitude]';
else
   f = [0 0]';
end
end

function F = fcn_F_traj(s,delay,duration,F_max)
% 6th Order Polynomial Function with Zero Initial and Final P,V,A
% At s = 0.5, the function reaches F_max
s = (s-delay)/duration;
F = F_max * s.^3 .* (-64*s.^3 + 192*s.^2 - 192*s + 64);
if F<0
   F = 0;
end

function X_plus = fcn_impact(X,p)
q_minus = X(1:3)';
dq_minus = X(4:6)';
params = p.params;

M = fcn_Me(q_minus,params);
Jhc = fcn_Jhc(q_minus,params); % Foot Relative to Hip, Hip Frame (I could've called the wrong variable)

% Impact Map Equations A * [ dq_plus ; F_impact ] = b
% M * (dq_plus - dq_minus) = J' * F_impact % Change in Momentum, i.e. Inelastic Collision
% J * dq_plus = 0   % Velocity of the Contact Point set to Zero
A_mat = [ M , -Jhc' ;
       Jhc, zeros(2,2)];
B_vec = [ M * dq_minus ; zeros(2,1) ];
sol_vec = A_mat \ B_vec;

X_plus = [ q_minus ; sol_vec(1:3) ]';
F_imp = sol_vec(4:5)';

end


%%
function [dXdt,u_out,F_out] = robot_dyn_aerial(t,X,p)
% sim_t = t; % Uncomment to Output Simulation Time during Runtime

params = p.params;

[m , n] = size(X);
if n == 6
   X = X'; % Change from Row to Column Vector
end
N = size(X,2);
u_out = zeros(N,2); % Two Force Inputs
F_out = zeros(N,2); % 2-D Force (X/Y)
dXdt = zeros(size(X,1),N);

% If Used in ODE45 Integrator, N = 1 - else N will be same length as input
for i = 1:N
   q = X(1:3,i);
   dq = X(4:6,i);

   % Leg Dynamics
   M = fcn_Me(q,params);
   C = fcn_Ce(q,dq,params);
```

```
    G = fcn_Ge(q,params);
    B = fcn_Be(q,params);
    J14 = fcn_J14(q,params);

    % Friction Modeling (INCOMPLETE)
    % tau_Coulomb = p.friction(1:3,1);
    % b_viscous = p.friction(4:6,1);
    % friction = - diag(tau_Coulomb)*tanh(20*dq) - diag(b_viscous)*dq;
    friction = [0 0 0]';

    % Controller Torque
    [tau,F] = controller_aerial(t(i),X(:,i),p);
    % Torque Saturation (INCOMPLETE)

    % Data Logging of Controller
    u_out(i,:) = tau(2:3)';
    F_out(i,:) = F';
    % External Forces
    f_app = [ 0 0 ]';

    % Belt Transmission Backlash (INCOMPLETE)

    % Dynamic Equations
    ddq = (M + p.M_rotor)\ (B * tau + J14' * f_app + friction - C*dq - G );
    dXdt(:,i) = [dq; ddq];
end

end

% contact = fcn_contact_check;
% f_contact = fcn_contact(t,X,p);
% f_disturb   = fcn_disturbance(t);
% f_contact = [0 0]';
% f_disturb = [0 0]';
% f_tot = f_contact + f_disturb;

function [dXdt,u_out,F_out,test] = robot_dyn_stance_jump(t,X,p,test)
% sim_t = t; % Uncomment to Output Simulation Time during Runtime

params = p.params;
% time_contact = p.time_contact(1,end);
% point_contact = p.point_contact;

[m , n] = size(X);
if n == 6
    X = X'; % Change from Row to Column Vector
end

N = size(X,2);
u_out = zeros(N,2); % Two Force Inputs
F_out = zeros(N,2); % 2-D Force (X/Y)
dXdt = zeros(size(X,1),N);

% If Used in ODE45 Integrator, N = 1 - else N will be same length as input
for i = 1:N
    q = X(1:3,i);
    dq = X(4:6,i);


    % Leg Dynamics
    M = fcn_Me(q,params);
    C = fcn_Ce(q,dq,params);
    G = fcn_Ge(q,params);
    B = fcn_Be(q,params);

    % Foot to Hip Jacobian
    J14 = fcn_J14(q,params);

    % Contact Constraint Jacobians
    Jhc = fcn_Jhc(q,params);
    dJhc = fcn_dJhc(q,dq,params);

    % Friction Modeling (INCOMPLETE)
    % tau_Coulomb = p.friction(1:3,1);
    % b_viscous = p.friction(4:6,1);
```

```
  % friction = - diag(tau_Coulomb)*tanh(20*dq) - diag(b_viscous)*dq;
  friction = [0 0 0]';

  % Controller Torque
  [tau,test] = controller_jump(t(i),X(:,i),p,test); % Feedforward Force + Feedback Stabilization

  % Torque Saturation (INCOMPLETE)

  % Data Logging of Controller
  u_out(i,:) = [ tau(2:3) ];
  % External Forces (s)
  f_app = [ 0 0 ]';

  % Constrained Dynamic Equations
  % A[ddq ; GRF] = B
  % Regular Dynamics w/ Applied Force: M*ddq - J'*GRF= (B * tau + J' * f_app + friction - C*dq - G );
  % 2nd Derivative of Non-Slip Holonomic Constraint : J * ddq + 0 * GRF =
  % -dJ * dq - Baumgarte Stabilization Terms

  % Baumgarte Stabilization to Prevent ODE solver from violating
  % Holonomic Constraint: dJhc*dq + Jhc*ddq + 2*alpha*Jhc*dq + beta^2*p_hc(0)
  % Stops it from drifting through the floor with another sort of tunable
  % PD control with parameters alpha and beta
  alpha = p.alpha;
  beta  = p.beta;

  A_matrix = [ (M + p.M_rotor) , -Jhc';
      Jhc, zeros(2,2)];
  B_vector = [ (B * tau + J14' * f_app + friction - C*dq - G ) ; % Re-Evaluate Jacobian in Front of f_app to see if the right type
          -dJhc * dq - 2 * alpha * Jhc * dq - beta^2 * fcn_phc(q,params)];
  v = A_matrix \ B_vector;
  ddq = v(1:3);
  F_out(i,:) = reshape(v(4:5),[1,2]);
  dXdt(:,i) = [dq; ddq];
end

end

function [dXdt,u_out,F_out,test] = robot_dyn_stance_stand(t,X,p,test)
% sim_t = t; % Uncomment to Output Simulation Time during Runtime

params = p.params;
% time_contact = p.time_contact(1,end);
% point_contact = p.point_contact;

[m , n] = size(X);
if n == 6
  X = X'; % Change from Row to Column Vector
end

N = size(X,2);
u_out = zeros(N,2); % Two Force Inputs
F_out = zeros(N,2); % 2-D Force (X/Y)
dXdt = zeros(size(X,1),N);

% If Used in ODE45 Integrator, N = 1 - else N will be same length as input
for i = 1:N
  q = X(1:3,i);
  dq = X(4:6,i);


  % Leg Dynamics
  M = fcn_Me(q,params);
  C = fcn_Ce(q,dq,params);
  G = fcn_Ge(q,params);
  B = fcn_Be(q,params);

  % Foot to Hip Jacobian
  J14 = fcn_J14(q,params);

  % Contact Constraint Jacobians
  Jhc = fcn_Jhc(q,params);
  dJhc = fcn_dJhc(q,dq,params);

  % Friction Modeling (INCOMPLETE)
```

```
    % tau_Coulomb = p.friction(1:3,1);
    % b_viscous = p.friction(4:6,1);
    % friction = - diag(tau_Coulomb)*tanh(20*dq) - diag(b_viscous)*dq;
    friction = [0 0 0]';

    % Controller Torque
    [tau] = controller_stand(t(i),X(:,i),p); % Feedforward Force + Feedback Stabilization

    % Torque Saturation (INCOMPLETE)

    % Data Logging of Controller
    u_out(i,:) = [ tau(2:3) ];
    % External Forces (s)
    f_app = [ 0 0 ]';

    % Constrained Dynamic Equations
    % A[ddq ; GRF] = B
    % Regular Dynamics w/ Applied Force: M*ddq - J'*GRF= (B * tau + J' * f_app + friction - C*dq - G );
    % 2nd Derivative of Non-Slip Holonomic Constraint : J * dqq + 0 * GRF = -dJ * dq
    % Baumgarte Stabilization to Prevent ODE solver from violating
    % Holonomic Constraint: dJhc*dq + Jhc*ddq + 2*alpha*Jhc*dq + beta^2*p_hc(0)
    % Stops it from drifting through the floor with another sort of tunable
    % PD control with parameters alpha and beta
    alpha = p.alpha_standing;
    beta  = p.beta_standing; % Needed to turn it up, but it affects the solver as well

    A_matrix = [ (M + p.M_rotor), -Jhc';
        Jhc, zeros(2,2)];

    B_vector = [ (B * tau + J14' * f_app + friction - C*dq - G ) ;
            -dJhc * dq - 2 * alpha * Jhc * dq - beta^2 * fcn_phc(q,params)];
    v = A_matrix \ B_vector;
    ddq = v(1:3);
    F_out(i,:) = reshape(v(4:5),[1,2]);
    dXdt(:,i) = [dq; ddq];
end

end


function [zeroCrossing,isterminal,direction] = event_contact(t,X,p)

params = p.params;
q = X(1:3);
p4 = fcn_p4(q,params);
% % Ensure Simulation Time does not affect
% if p4(2)<=0
%     p4(2)=0;
% end
% Assumptions of Flat Ground
zeroCrossing = p4(2); % Y-coordinate
isterminal = 1; % Results in termination when condition met
direction = -1; % Can Occur when from Positive or Negative Crossing

function [zeroCrossing,isterminal,direction] = event_flight(t,X,p)

params = p.params;
test.x = zeros(0);
test.F = zeros(3,0);
test.tau = zeros(3,0);



q = X(1:3,:);
dq = X(4:6,:);

% Leg Dynamics
M = fcn_Me(q,params);
C = fcn_Ce(q,dq,params);
G = fcn_Ge(q,params);
B = fcn_Be(q,params);

% Foot to Hip Jacobian
J14 = fcn_J14(q,params);
```

```
% Contact Constraint Jacobians
Jhc = fcn_Jhc(q,params);
dJhc = fcn_dJhc(q,dq,params);

% Friction Modeling (INCOMPLETE)
% tau_Coulomb = p.friction(1:3,1);
% b_viscous = p.friction(4:6,1);
% friction = - diag(tau_Coulomb)*tanh(20*dq) - diag(b_viscous)*dq;
friction = [0 0 0]';

% Controller Torque
tau = controller_jump(t,X,p,test); % Feedforward Force + Feedback Stabilization

% Torque Saturation (INCOMPLETE)

% External Forces (s)
f_app = [ 0 0 ]';

% Constrained Dynamic Equations
% A[ddq ; GRF] = B
% Regular Dynamics w/ Applied Force: M*ddq - J'*GRF= (B * tau + J' * f_app + friction - C*dq - G );
% 2nd Derivative of Non-Slip Holonomic Constraint : J * ddq + 0 * GRF =
% -dJ * dq - Baumgarte Stabilization Terms

% Baumgarte Stabilization to Prevent ODE solver from violating
% Holonomic Constraint: dJhc*dq + Jhc*ddq + 2*alpha*Jhc*dq + beta^2*p_hc(0)
% Stops it from drifting through the floor with another sort of tunable
% PD control with parameters alpha and beta
alpha = p.alpha;
beta  = p.beta;

A_matrix = [ (M + p.M_rotor), -Jhc';
    Jhc, zeros(2,2)];
B_vector = [ (B * tau + J14' * f_app + friction - C*dq - G ) ;
        -dJhc * dq - 2 * alpha * Jhc * dq - beta^2 * fcn_phc(q,params)];
v = A_matrix \ B_vector;
ddq = v(1:3);
GRF = reshape(v(4:5),[1,2]);

GRFy = GRF(2);
threshold = p.threshold; % 2.5% of Static Weight


% Assumptions of Flat Ground
zeroCrossing = GRFy - threshold;  % Y-Force > Some Threshhold (<< Static Weight)
isterminal = 1; % Results in termination when condition met
direction = -1; % Can Occur when from Positive or Negative Crossing

end

% Generate the Dynamics for the Leg
% Adapted from Yanran Ding's Script
clear all
syms q1 q2 q3 real
syms dq1 dq2 dq3 real
syms l1 l2 l3 real
syms M1 M2 M3 real
syms J1_xx J1_yy J1_zz J1_xy J1_xz J1_yz real
syms J2_xx J2_yy J2_zz J2_xy J2_xz J2_yz real
syms J3_xx J3_yy J3_zz J3_xy J3_xz J3_yz real
syms g real
syms X Y real
syms l2comx l2comy l3comx l3comy real

%% Variable List

% Params
m_list_params = {
   'g'  'p(1)'
   'l1' 'p(2)';
   'l2' 'p(3)';
   'l3' 'p(4)';
   'M1' 'p(5)';
   'M2' 'p(6)';
   'M3' 'p(7)';
```

```
  'J1_xx' 'p(8)';
  'J1_yy' 'p(9)';
  'J1_zz' 'p(10)';
  'J1_xy' 'p(11)';
  'J1_xz' 'p(12)';
  'J1_yz' 'p(13)';
  'J2_xx' 'p(14)';
  'J2_yy' 'p(15)';
  'J2_zz' 'p(16)';
  'J2_xy' 'p(17)';
  'J2_xz' 'p(18)';
  'J2_yz' 'p(19)';
  'J3_xx' 'p(20)';
  'J3_yy' 'p(21)';
  'J3_zz' 'p(22)';
  'J3_xy' 'p(23)';
  'J3_xz' 'p(24)';
  'J3_yz' 'p(25)';
  'l2comx' 'p(26)'
  'l2comy' 'p(27)'
  'l3comx' 'p(28)'
  'l3comy' 'p(29)'};

% joint position
m_list_q = {
  'q1' 'q(1)';
  'q2' 'q(2)';
  'q3' 'q(3)'};

% joint velocity
m_list_dq = {
  'dq1' 'dq(1)';
  'dq2' 'dq(2)';
  'dq3' 'dq(3)'};

% task space position
m_list_p = {
  'X' 'pos(1)';
  'Y' 'pos(2)'};

%% Joint Variables
q = [q1 q2 q3]'; % height, theta1, theta2
dq = [dq1 dq2 dq3]';

%% IK - 2D [From Base to Foot]
pos = [X ; Y];

length_int = norm(pos);
theta_2 = pi - acos((l2^2 + l3^2 - length_int^2)/(2*l2*l3));
theta_1 = atan2( Y , X ) - atan2(l3*sin(theta_2),(l2+l3*cos(theta_2))) + pi/2;
assume([X Y l2 l3],'real');
q_IK = [ theta_1 ; theta_2 ];
q_IK = simplify(q_IK,'Steps',50);
write_fcn_m('fcn_IK.m',{'pos','p'},[m_list_p;m_list_params],{q_IK,'q_IK'});

%% FK - 2D

T01 = [eye(2), [0,q1]';
     0 0 1];
T12 = [rot(q2-pi/2), [l1,0]';
     0 0 1];
T23 = [rot(q3), [l2,0]';
     0 0 1];
T34 = [eye(2) , [l3,0]';
     0 0 1];
write_fcn_m('fcn_T01.m',{'q','p'},[m_list_q;m_list_params],{T01,'T01'});
write_fcn_m('fcn_T12.m',{'q','p'},[m_list_q;m_list_params],{T12,'T12'});
write_fcn_m('fcn_T23.m',{'q','p'},[m_list_q;m_list_params],{T23,'T23'});
write_fcn_m('fcn_T34.m',{'q','p'},[m_list_q;m_list_params],{T34,'T34'});
% Joint Positions

T02 = T01 * T12;
p2 = T02(1:2,3);
J2 = jacobian(p2,q);
write_fcn_m('fcn_p2.m',{'q','p'},[m_list_q;m_list_params],{p2,'p2'});
```

```
write_fcn_m('fcn_J2.m',{'q','p'},[m_list_q;m_list_params],{J2,'J2'});

T03 = T02 * T23;
p3 = T03(1:2,3);
write_fcn_m('fcn_p3.m',{'q','p'},[m_list_q;m_list_params],{p3,'p3'});

T04 = T03 * T34;
p4 = T04(1:2,3);
J4 = jacobian(p4,q);
write_fcn_m('fcn_p4.m',{'q','p'},[m_list_q;m_list_params],{p4,'p4'});
write_fcn_m('fcn_J4.m',{'q','p'},[m_list_q;m_list_params],{J4,'J4'});

% Foot w.r.t to Base of Prismatic
T14 = T12*T23*T34;
p14 = T14(1:2,3);
J14 = jacobian(p14,q);
write_fcn_m('fcn_p14.m',{'q','p'},[m_list_q;m_list_params],{p14,'p14'});
write_fcn_m('fcn_J14.m',{'q','p'},[m_list_q;m_list_params],{J14,'J14'});

% Foot w.r.t to Hip Joint
T24 = T23 * T34;
p24 = T24(1:2,3);
J24 = jacobian(p24,q);
write_fcn_m('fcn_p24.m',{'q','p'},[m_list_q;m_list_params],{p24,'p24'});
write_fcn_m('fcn_J24.m',{'q','p'},[m_list_q;m_list_params],{J24,'J24'});


% Jacobian Derivative
dJ4 = sym('dJ4',size(J4));
for ii = 1:size(dJ4,2)
   dJ4(:,ii) = jacobian(J4(:,ii),q) * dq;
end
write_fcn_m('fcn_dJ4.m',{'q','dq','p'},[m_list_q;m_list_dq;m_list_params],{dJ4,'dJ4'});

%% Foot Constraints

% Point of Contact is the toe position at impact time
point_contact = [0 , 0 , 0]';
z_hat = [ 0 0 1 ]';
y_hat = [ 0 1 0 ]';
x_hat = [ 1 0 0 ]';
R = [ x_hat y_hat z_hat];

%%
% The Holonomic constraints are:
% foot position x and y in Toe Frame do not change during stance
hol_con = p4([1,2]);
Jhc = jacobian(hol_con,q);
dJhc = sym('dJhc',size(Jhc));
for ii = 1:size(Jhc,2)
   dJhc(:,ii) = jacobian(Jhc(:,ii),q) * dq;
end
write_fcn_m('fcn_phc.m',{'q','p'},[m_list_q;m_list_params],{hol_con,'phc'})
write_fcn_m('fcn_Jhc.m',{'q','p'},[m_list_q;m_list_params],{Jhc,'Jhc'});
write_fcn_m('fcn_dJhc.m',{'q', 'dq', 'p'},[m_list_q;m_list_dq;m_list_params],{dJhc,'dJhc'});

%% COM Kinematics
T1com = [eye(2), [l1/2,0]';
     0 0 1];
T2com = [eye(2), [l2comx,l2comy]';
     0 0 1];
T3com = [eye(2) , [l3comx,l3comy]';
     0 0 1];

T01com = T01*T1com;
R01com = T01com(1:2,1:2);
p01com = T01com(1:2,3);
v01com = jacobian(p01com,q) * dq;

T02com = T01*T12*T2com;
R02com = T02com(1:2,1:2);
p02com = T02com(1:2,3);
v02com = jacobian(p02com,q) * dq;

T03com = T01*T12*T23*T3com;
```

```
R03com = T03com(1:2,1:2);
p03com = T03com(1:2,3);
v03com = jacobian(p03com,q) * dq;

%% Angular Velocities
% Rotation Axis
k1 = [0 0 0]';
k2 = [0 0 1]';
k3 = [0 0 1]';

Jw = [ [R01com [0 ,0]'; 0 0 1]*k1 , [R02com [0 ,0]'; 0 0 1]*k2 , [R03com [0 ,0]'; 0 0 1]*k3 ];

omega1 = Jw(:,1) * dq(1);
omega2 = Jw(:,1:2) * dq(1:2);
omega3 = Jw(:,1:3) * dq(1:3);

%% Energy
% Inertia of Links
I1 = [[J1_xx J1_xy J1_xz]; [J1_xy J1_yy J1_yz]; [J1_xz J1_yz J1_zz];];
I2 = [[J2_xx J2_xy J2_xz]; [J2_xy J2_yy J2_yz]; [J2_xz J2_yz J2_zz];];
I3 = [[J3_xx J3_xy J3_xz]; [J3_xy J3_yy J3_yz]; [J3_xz J3_yz J3_zz];];

R01com3d = [R01com [0 ,0]'; 0 0 1];
R02com3d = [R02com [0 ,0]'; 0 0 1];
R03com3d = [R03com [0 ,0]'; 0 0 1];

KE_1 = 0.5 * v01com' * M1 * v01com + 0.5 * omega1' * (R01com3d*I1*R01com3d') * omega1;
KE_2 = 0.5 * v02com' * M2 * v02com + 0.5 * omega2' * (R02com3d*I2*R02com3d') * omega2;
KE_3 = 0.5 * v03com' * M3 * v03com + 0.5 * omega3' * (R03com3d*I3*R03com3d') * omega3;

KE = simplify(KE_1 + KE_2 + KE_3);

PE = g * ( M1 * p01com(2) + M2 * p02com(2) + M3 * p03com(2));

upsilon = [0 q2 q3];

%% Euler Lagrange Function
[Me, Ce, Ge, Be] = std_dynamics(KE,PE,q,dq,upsilon);

write_fcn_m('fcn_Me.m',{'q', 'p'},[m_list_q;m_list_params],{Me,'Me'});
write_fcn_m('fcn_Ce.m',{'q', 'dq', 'p'},[m_list_q;m_list_dq;m_list_params],{Ce,'Ce'});
write_fcn_m('fcn_Ge.m',{'q', 'p'},[m_list_q;m_list_params],{Ge,'Ge'});
write_fcn_m('fcn_Be.m',{'q', 'p'},[m_list_q;m_list_params],{Be,'Be'});

%% Change of Base Grav Comp ( From Eric )
% l3comx_stance = l3-l3comx;
% l3comy_stance = l3comy;
% l2comx_stance = l2-l2comx;
% l2comy_stance = l2comy;
% l2com_stance = sqrt(l2comx_stance^2 + l2comy_stance^2);
% q1_stance = pi/2 + q2 + q1;
% q2_stance = -q2;
% FK_hip = [l3*cos(q1_stance) + l2*cos(q1_stance+q2_stance) ;
%         l3*sin(q1_stance) + l2*sin(q1_stance+q2_stance)];
% FK_tibia_com = [l3comx_stance*cos(q1_stance);
%             l3comx_stance*sin(q1_stance);];
% FK_femur_com = [l3*cos(q1_stance) + l2comx_stance*cos(q1_stance+q2_stance);
%              l3*sin(q1_stance) + l2comx_stance*sin(q1_stance+q2_stance); ];
% G_comp_stance = [0;
%     -(M3*g*l3comx_stance*cos(q1_stance) + M1*g*(l2*cos(q1_stance+q2_stance)+l3*cos(q1_stance)) +M2*g*(l2com_stance*cos(q1_stance+q2_stance-
hip_angle)+l3*cos(q1_stance)));
%     -(M1+M2+M3)*g*l3*cos(q1+q2) + M3*g*l3comx*cos(q1+q2)];



%% Rotation Functions
function R = rot(theta)
R = [cos(theta)    -sin(theta)   ;
    sin(theta)    cos(theta)  ];

end

%%author: Ben Morris and Eric Westervelt
function [D,C,G,B] = std_dynamics(KE,PE,x,dx, xrel) % all variables must be symbolic when passed, symbolic variables are returned
                G=jacobian(PE,x).';
```

```
            G=simple_elementwise(G);

            tem=jacobian(KE,dx).'; % tem=simple(jacobian(KE,dx).');
            D=simple_elementwise(jacobian(tem,dx)); %simple(jacobian(tem,dx));

            syms C
            n=max(size(x));
            for k=1:n
                        for j=1:n
                                    C(k,j)=0;
                                    for i=1:n
                                                C(k,j)=C(k,j)+(1/2)*(diff(D(k,j),x(i))+diff(D(k,i),x(j))-diff(D(i,j),x(k)))*dx(i);
                                    end
                        end
            end
            C=simple_elementwise(C);%simple(C);

  B = jacobian(xrel,x)' ;
end

% Function to perform simple() one element at a time
function M = simple_elementwise(M)
  for i=1:size(M,1)
    for j=1:size(M,2)
      M(i,j) = simplify(M(i,j)) ;
    end
  end
end

% From Yanran Ding to Turn Angle-Varying Values into Functions
function write_fcn(fcn_name,arguments,replace_list,list)

[pathstr, name, ext] = fileparts(fcn_name) ;
disp(['- writing ' name ext]);

% open file and write first line

fid = fopen(fcn_name,'w');
fprintf(fid,'function [');

for item = 1:1:size(list,1)
 currentname = list{item,2};
 if (item > 1) fprintf(fid,',');
 end
 fprintf(fid,'%s',currentname);
end

[path,name,ext] = fileparts(fcn_name);
fprintf(fid,'] = %s(',name);

for item = 1:1:size(arguments,2)
 % write arguments
 currentname = arguments{1,item};
 if (item > 1) fprintf(fid,',');
 end
 fprintf(fid,'%s',currentname);
end
fprintf(fid,')\n\n');
for ii=1:size(list,1)
  [n m] = size(list{ii,1});
  fprintf(fid,'%s = zeros(%d,%d);\n\n', list{ii,2}, n, m);
end

%fprintf(fid,'  model_params;\n\n');

for item = 1:1:size(list,1)
 % write variables to file
 currentvar = list{item,1};
 currentname = list{item,2};
 [n,m]=size(currentvar);
 for i=1:n
  for j=1:m
   Temp0=(currentvar(i,j));
   if (isnumeric(Temp0))
    Temp1 = num2str(Temp0);
```

```
    else
     Temp1=replace(char(Temp0),replace_list);
    end
    Temp2=['  ',currentname,'(',num2str(i),',',num2str(j),')=',Temp1,';'];
    fixlength_m(Temp2,'*+',100,'        ',fid);     fprintf(fid,'\n');
  end
 end
 fprintf(fid,'\n%s',' ');
end
status = fclose(fid);
%disp(' - done');

%=============================================
% function str = replace(str,replace_list)
% % process the longest strings first to keep from accidentally replacing substrings of longer strings
% % perform a bubble sort on 'replace_list' according to decending length of the first element
% % <del>
% % if (size(replace_list,1) > 0)
% %   for loop=1:1:size(replace_list,1)
% %    for i=1:1:size(replace_list,1)-1
% %      if (length(replace_list{i,1}) < length(replace_list{i+1,1}))
% %        temp = replace_list(i,:);
% %        replace_list(i,:) = replace_list(i+1,:);
% %        replace_list(i+1,:) = temp;
% %      end
% %    end
% %   end
% %   for i=size(replace_list,1):-1:1
% %    orig_str = char(replace_list(i,1));
% %    new_str = char(replace_list(i,2));
% %    str = strrep(str,orig_str,new_str);
% %   end
% % end
% % </del>
% 
% % <add>
% delimiters = ' +-/*^%&|!()' ;
% rem = str ;
% str = '' ;
% while(length(rem))
%    % Handle leading delimiters
%    if(length(findstr(rem(1), delimiters))) % First character is a delimiter
%       str = [str rem(1)] ;
%       rem(1) = [] ;
%       continue ;
%    end
%    [tok rem] = strtok(rem, delimiters) ;
%    if(length(tok))
%       found = false ;
%       for i=1:size(replace_list,1)
%          if(strcmp(tok, char(replace_list(i,1))))
%             str = [str char(replace_list(i,2))] ; % Replace token with replacement string and concatenate.
%             found = true ;
%             break ;
%          end
%       end
%       if(~found)
%          str = [str tok] ; % Concatenate token to the string if token is not being replaced with another string.
%       end
%    end
%    if(length(rem)) % If more to process
%       str = [str rem(1)] ; % Add found delimiter to the string
%       rem(1) = [] ; % Remove delimiter
%    end
% end
% % </add>
function str = replace(str, replace_list)
  for i = 1:size(replace_list, 1)
     orig = ['(?<![A-Za-z0-9_])', replace_list{i,1}, '(?![A-Za-z0-9_])']; % Match whole word only
     new = replace_list{i,2};
     str = regexprep(str, orig, new);
  end
```

```
function fixlength_m(s1,s2,len,indent,fid)

%FIXLENGTH Returns a string which has been divided up into < LEN
%character chuncks with the '...' line divide appended at the end%of each chunck.
%   FIXLENGTH(S1,S2,L) is string S1 with string S2 used as break points into
%   chuncks less than length L.
%Eric Westervelt%5/31/00
%1/11/01 - updated to use more than one dividing string
%4/29/01 - updated to allow for an indent
%3/24/05 - bjm - non-iterative operation, much faster for large files

A=[];
for c = 1:length(s2)
  A = [A findstr(s1,s2(c))];
  A = sort(A);
end
if (length(A)>0)
  dA = [diff(A)];
  B = A;
  runlen = A(1);
  for i=1:1:length(dA)
    if (runlen + dA(i) <= len)
      runlen = runlen + dA(i);
      B(i) = 0;
    else
      runlen = 0;
    end
  end
  B(end) = 0;
  % don't make a newline after the last entry
  B=[0 B(B>0) length(s1)];
  for i=1:1:length(B)-1
    if (i==1)
      fprintf(fid,'%s',s1(B(i)+1:B(i+1)));
    else
      fprintf(fid,'...\n%s%s',indent,s1(B(i)+1:B(i+1)));
    end
  end
else
  fprintf(fid,'%s',s1);
end

function [T34] = fcn_T34(q,p)

T34 = zeros(3,3);

  T34(1,1)=1;
  T34(1,2)=0;
  T34(1,3)=p(4);
  T34(2,1)=0;
  T34(2,2)=1;
  T34(2,3)=0;
  T34(3,1)=0;
  T34(3,2)=0;
  T34(3,3)=1;

function [T23] = fcn_T23(q,p)

T23 = zeros(3,3);

  T23(1,1)=cos(q(3));
  T23(1,2)=-sin(q(3));
  T23(1,3)=p(3);
  T23(2,1)=sin(q(3));
  T23(2,2)=cos(q(3));
  T23(2,3)=0;
  T23(3,1)=0;
  T23(3,2)=0;
  T23(3,3)=1;

 function [T12] = fcn_T12(q,p)

T12 = zeros(3,3);

  T12(1,1)=cos(q(2) - pi/2);
```

```
 T12(1,2)=-sin(q(2) - pi/2);
 T12(1,3)=p(2);
 T12(2,1)=sin(q(2) - pi/2);
 T12(2,2)=cos(q(2) - pi/2);
 T12(2,3)=0;
 T12(3,1)=0;
 T12(3,2)=0;
 T12(3,3)=1;

function [T03] = fcn_T03(q,p)

T03 = zeros(4,4);

 T03(1,1)=cos(q(1))*cos(q(2));
 T03(1,2)=cos(q(1))*sin(q(2))*sin(q(3)) - cos(q(3))*sin(q(1));
 T03(1,3)=sin(q(1))*sin(q(3)) + cos(q(1))*cos(q(3))*sin(q(2));
 T03(1,4)=p(3)*cos(q(1))*cos(q(2)) - p(4)*sin(q(1)) - p(5)*sin(q(1))*sin(q(3)) - p(5)*cos(q(1))*...
     cos(q(3))*sin(q(2));
 T03(2,1)=cos(q(2))*sin(q(1));
 T03(2,2)=cos(q(1))*cos(q(3)) + sin(q(1))*sin(q(2))*sin(q(3));
 T03(2,3)=cos(q(3))*sin(q(1))*sin(q(2)) - cos(q(1))*sin(q(3));
 T03(2,4)=p(4)*cos(q(1)) + p(3)*cos(q(2))*sin(q(1)) + p(5)*cos(q(1))*sin(q(3)) - p(5)*cos(q(3))*...
     sin(q(1))*sin(q(2));
 T03(3,1)=-sin(q(2));
 T03(3,2)=cos(q(2))*sin(q(3));
 T03(3,3)=cos(q(2))*cos(q(3));
 T03(3,4)=p(2) - p(3)*sin(q(2)) - p(5)*cos(q(2))*cos(q(3));
 T03(4,1)=0;
 T03(4,2)=0;
 T03(4,3)=0;
 T03(4,4)=1;

function [T02] = fcn_T02(q,p)

T02 = zeros(4,4);

 T02(1,1)=cos(q(1))*cos(q(2));
 T02(1,2)=-sin(q(1));
 T02(1,3)=cos(q(1))*sin(q(2));
 T02(1,4)=p(3)*cos(q(1))*cos(q(2)) - p(4)*sin(q(1));
 T02(2,1)=cos(q(2))*sin(q(1));
 T02(2,2)=cos(q(1));
 T02(2,3)=sin(q(1))*sin(q(2));
 T02(2,4)=p(4)*cos(q(1)) + p(3)*cos(q(2))*sin(q(1));
 T02(3,1)=-sin(q(2));
 T02(3,2)=0;
 T02(3,3)=cos(q(2));
 T02(3,4)=p(2) - p(3)*sin(q(2));
 T02(4,1)=0;
 T02(4,2)=0;
 T02(4,3)=0;
 T02(4,4)=1;

function [T01] = fcn_T01(q,p)

T01 = zeros(3,3);

 T01(1,1)=1;
 T01(1,2)=0;
 T01(1,3)=0;
 T01(2,1)=0;
 T01(2,2)=1;
 T01(2,3)=q(1);
 T01(3,1)=0;
 T01(3,2)=0;
 T01(3,3)=1;

function [phc] = fcn_phc(q,p)

phc = zeros(2,1);

 phc(1,1)=p(2) + p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*cos(q(2) - pi/2);
 phc(2,1)=q(1) + p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) + p(3)*sin(q(2) - pi/2);

function [p24] = fcn_p24(q,p)
```

```
p24 = zeros(2,1);

 p24(1,1)=p(3) + p(4)*cos(q(3));
 p24(2,1)=p(4)*sin(q(3));

function [p14] = fcn_p14(q,p)

p14 = zeros(2,1);

 p14(1,1)=p(2) + p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*cos(q(2) - pi/2);
 p14(2,1)=p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) + p(3)*sin(q(2) - pi/2);


function [p4] = fcn_p4(q,p)

p4 = zeros(2,1);

 p4(1,1)=p(2) + p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*cos(q(2) - pi/2);
 p4(2,1)=q(1) + p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) + p(3)*sin(q(2) - pi/2);


function [p3] = fcn_p3(q,p)

p3 = zeros(2,1);

 p3(1,1)=p(2) + p(3)*cos(q(2) - pi/2);
 p3(2,1)=q(1) + p(3)*sin(q(2) - pi/2);


function [p2] = fcn_p2(q,p)

p2 = zeros(2,1);

 p2(1,1)=p(2);
 p2(2,1)=q(1);

function [p1] = fcn_p1(q,p)

p1 = zeros(3,1);

 p1(1,1)=0;
 p1(2,1)=0;
 p1(3,1)=p(2);

function [Me] = fcn_Me(q,p)

Me = zeros(3,3);

 Me(1,1)=p(5) + p(6) + p(7);
 Me(1,2)=p(7)*p(29)*cos(q(2) + q(3)) + p(7)*p(28)*sin(q(2) + q(3)) + p(6)*p(27)*cos(q(2)) + p(7)*...
     p(3)*sin(q(2)) + p(6)*p(26)*sin(q(2));
 Me(1,3)=p(7)*p(29)*cos(q(2) + q(3)) + p(7)*p(28)*sin(q(2) + q(3));
 Me(2,1)=p(7)*p(29)*cos(q(2) + q(3)) + p(7)*p(28)*sin(q(2) + q(3)) + p(6)*p(27)*cos(q(2)) + p(7)*...
     p(3)*sin(q(2)) + p(6)*p(26)*sin(q(2));
 Me(2,2)=p(16) + p(22) + p(7)*p(3)^2 + p(6)*p(26)^2 + p(6)*p(27)^2 + p(7)*p(28)^2 + p(7)*p(29)^2 +...
      2*p(7)*p(3)*p(28)*cos(q(3)) - 2*p(7)*p(3)*p(29)*sin(q(3));
 Me(2,3)=p(22) + p(7)*p(28)^2 + p(7)*p(29)^2 + p(7)*p(3)*p(28)*cos(q(3)) - p(7)*p(3)*p(29)*sin(q(3));
 Me(3,1)=p(7)*p(29)*cos(q(2) + q(3)) + p(7)*p(28)*sin(q(2) + q(3));
 Me(3,2)=p(22) + p(7)*p(28)^2 + p(7)*p(29)^2 + p(7)*p(3)*p(28)*cos(q(3)) - p(7)*p(3)*p(29)*sin(q(3));
 Me(3,3)=p(22) + p(7)*p(28)^2 + p(7)*p(29)^2;


function [Jhc] = fcn_Jhc(q,p)

Jhc = zeros(2,3);

 Jhc(1,1)=0;
 Jhc(1,2)=- p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) - p(3)*sin(q(2) - pi/2);
 Jhc(1,3)=-p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));
 Jhc(2,1)=1;
 Jhc(2,2)=p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*cos(q(2) - pi/2);
 Jhc(2,3)=p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));

function [J24] = fcn_J24(q,p)
```

```
J24 = zeros(2,3);

 J24(1,1)=0;
 J24(1,2)=0;
 J24(1,3)=-p(4)*sin(q(3));
 J24(2,1)=0;
 J24(2,2)=0;
 J24(2,3)=p(4)*cos(q(3));

 function [J14] = fcn_J14(q,p)

J14 = zeros(2,3);

 J14(1,1)=0;
 J14(1,2)=- p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) - p(3)*sin(q(2) - pi/2);
 J14(1,3)=-p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));
 J14(2,1)=0;
 J14(2,2)=p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*cos(q(2) - pi/2);
 J14(2,3)=p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));


function [J4] = fcn_J4(q,p)

J4 = zeros(2,3);

 J4(1,1)=0;
 J4(1,2)=- p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) - p(3)*sin(q(2) - pi/2);
 J4(1,3)=-p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));
 J4(2,1)=1;
 J4(2,2)=p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*cos(q(2) - pi/2);
 J4(2,3)=p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));


function [J2] = fcn_J2(q,p)

J2 = zeros(2,3);

 J2(1,1)=0;
 J2(1,2)=0;
 J2(1,3)=0;
 J2(2,1)=1;
 J2(2,2)=0;
 J2(2,3)=0;

 function [q_IK] = fcn_IK(pos,p)

q_IK = zeros(2,1);

 q_IK(1,1)=pi/2 - atan2(p(4)*(1 - (abs(pos(1))^2 + abs(pos(2))^2 - p(3)^2 - p(4)^2)^2/(4*p(3)^2*...
     p(4)^2))^(1/2), p(3) + (abs(pos(1))^2 + abs(pos(2))^2 - p(3)^2 - p(4)^2)/(2*p(3))) + atan2(pos(2), pos(1));
 q_IK(2,1)=acos((pos(1)^2 + pos(2)^2 - p(3)^2 - p(4)^2)/(2*p(3)*p(4)));


function [Ge] = fcn_Ge(q,p)

Ge = zeros(3,1);

 Ge(1,1)=p(1)*(p(5) + p(6) + p(7));
 Ge(2,1)=p(1)*(p(7)*p(29)*cos(q(2) + q(3)) + p(7)*p(28)*sin(q(2) + q(3)) + p(6)*p(27)*cos(q(2)) +...
      p(7)*p(3)*sin(q(2)) + p(6)*p(26)*sin(q(2)));
 Ge(3,1)=p(7)*p(1)*(p(29)*cos(q(2) + q(3)) + p(28)*sin(q(2) + q(3)));

 function [dJhc] = fcn_dJhc(q,dq,p)

dJhc = zeros(2,3);

 dJhc(1,1)=0;
 dJhc(1,2)=- dq(2)*(p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*...
     cos(q(2) - pi/2)) - dq(3)*p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));
 dJhc(1,3)=- dq(2)*p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) - dq(3)*p(4)*...
     (cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));
 dJhc(2,1)=0;
 dJhc(2,2)=- dq(2)*(p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) + p(3)*...
     sin(q(2) - pi/2)) - dq(3)*p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));
```

```
 dJhc(2,3)=- dq(2)*p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) - dq(3)*p(4)*...
      (cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));
function [dJ4] = fcn_dJ4(q,dq,p)

dJ4 = zeros(2,3);

 dJ4(1,1)=0;
 dJ4(1,2)=- dq(2)*(p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) + p(3)*...
      cos(q(2) - pi/2)) - dq(3)*p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));
 dJ4(1,3)=- dq(2)*p(4)*(cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2)) - dq(3)*p(4)*...
      (cos(q(3))*cos(q(2) - pi/2) - sin(q(3))*sin(q(2) - pi/2));
 dJ4(2,1)=0;
 dJ4(2,2)=- dq(2)*(p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) + p(3)*...
      sin(q(2) - pi/2)) - dq(3)*p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));
 dJ4(2,3)=- dq(2)*p(4)*(cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3))) - dq(3)*p(4)*...
      (cos(q(3))*sin(q(2) - pi/2) + cos(q(2) - pi/2)*sin(q(3)));

 function [Ce] = fcn_Ce(q,dq,p)

Ce = zeros(3,3);

 Ce(1,1)=0;
 Ce(1,2)=dq(2)*(p(7)*p(28)*cos(q(2) + q(3)) - p(7)*p(29)*sin(q(2) + q(3)) + p(7)*p(3)*cos(q(2)) +...
       p(6)*p(26)*cos(q(2)) - p(6)*p(27)*sin(q(2))) + p(7)*dq(3)*(p(28)*cos(q(2) + q(3)) - p(29)*sin(q(2) + q(3)));
 Ce(1,3)=p(7)*(dq(2) + dq(3))*(p(28)*cos(q(2) + q(3)) - p(29)*sin(q(2) + q(3)));
 Ce(2,1)=0;
 Ce(2,2)=-p(7)*dq(3)*p(3)*(p(29)*cos(q(3)) + p(28)*sin(q(3)));
 Ce(2,3)=-p(7)*p(3)*(dq(2) + dq(3))*(p(29)*cos(q(3)) + p(28)*sin(q(3)));
 Ce(3,1)=0;
 Ce(3,2)=p(7)*dq(2)*p(3)*(p(29)*cos(q(3)) + p(28)*sin(q(3)));
 Ce(3,3)=0;

function [Be] = fcn_Be(q,p)

Be = zeros(3,3);

 Be(1,1)=0;
 Be(1,2)=0;
 Be(1,3)=0;
 Be(2,1)=0;
 Be(2,2)=1;
 Be(2,3)=0;
 Be(3,1)=0;
 Be(3,2)=0;
 Be(3,3)=1;

 import numpy as np
from pybear import Manager

import numpy as np
from scipy import constants
from math import sin, cos, acos, pi, sqrt, radians
from conrad_functions import *

class ConradController:
   def __init__(self, l1 = 0.125, l2 = 0.215, l1x = 0.01416, l2x = 0.1171, l1y = 0.05920, l2y = -0.01922, m0 = 0.649, m1 = 0.478, m2 = 0.0633):
      self.l1 = l1
      self.l2 = l2
      self.l1x = l1x
      self.l2x = l2x
      self.l1y = l1y
      self.l2y = l2y
      self.m0 = m0  # Mass of base (before joint 1)
      self.m1 = m1  # Mass of link 1 assembly
      self.m2 = m2  # Mass of link 2 assembly (including foot contact switch)
      % the offset angles/zeros, add pi/2 to q1 to account for the BEAR's default orientation

   # Call this function and get the control torques in return
   def calc_control_torque(self, desired, q_rad, qd_rad_s, Kp, Kd, current_state,leg):
      # q_error = q_des_rad - q_rad
      q_des_rad = q_des_rad - [-pi/2, 0]  # Adjust q1 to BEAR's default orientation
      if current_state == 'AERIAL':
         # In AERIAL phase, we use the joint space PD control
         J = function_J14(q_rad, self.l1, self.l2)
         x_error = desired - function_p14(q_rad, self.l1, self.l2)
```

```
        x_dot = J @ qd_rad_s
        u = grav_comp_torques + friction_comp_torques + Kp @ x_error - Kd @ qd_rad_s
        # Task Space PD control with gravity and friction compensation
        grav_comp_torques = function_G(q_rad, constants.g, self.m1,self.m2, self.l1x, self.l1y, self.l2x,self.l2y,self.l1,self.l2)
        friction_comp_torques = self.calc_friction_torque(qd_rad_s)
        f = Kp @ x_error - Kd @ x_dot
        u = grav_comp_torques + friction_comp_torques + J.T @ f
      elif current_state == 'STANCE':
        # In STANCE phase, we use the task space PD control
        x_error = desired - function_p14(q_rad, self.l1, self.l2)
        J = function_J14(q_rad, self.l1, self.l2)
        x_dot = J @ qd_rad_s
        grav_comp_torques = function_G(q_rad, constants.g, self.m1,self.m2, self.l1x, self.l1y, self.l2x,self.l2y,self.l1,self.l2)
        f_ff = constants.g * np.array([0, (self.m0+self.m1+self.m2)])  # Static weight force feedforward
        friction_comp_torques = self.calc_friction_torque(qd_rad_s)
        f = Kp @ x_error - Kd @ x_dot
        u = grav_comp_torques + friction_comp_torques + J.T @ f
      return u
```

```
function [u,test] = controller_jump(t,X,p,test)
params = p.params;
q = X(1:3);
dq = X(4:6);
G = fcn_Ge(q,params);

% Control Gains in the Ground (Task Space)
Kp = p.Kp_stance;
Kd = p.Kd_stance;

% Feedback Torque using Controller to Stabilize System About Set Point in Task Space
x_des = p.foot_d_stance; % Desired Task Space Position of the Foot w.r.t Base
x = fcn_p14(q,params);
% Force Feedforward (Static Weight)
f_ff = [0;p.weight];
% Jacobian from Hip to Foot
J14 = fcn_J14(q,params);
x_dot = J14*dq;
% q_des = fcn_IK(x_des,params);
f_pd = Kp * (x_des - x) - Kd * x_dot;
u = (J14' * f_pd) -(J14' * f_ff) + G; % Gravity Compensation to Reduce SS Errors
```

```
  def calc_friction_torque(self, qd_rad_s):
    minimum_velocity = (1.08 + 3.32 + 0.86) / 3  # rad/s
    slope_near_zero = (0.192 + 0.113 + 0.243) / 3

    viscous_pos = (0.045 + 0.034 + 0.027) * 0.1 / 3
    viscous_neg = (0.55 + 0.032 + 0.024) * 0.1 / 3
    coulomb_pos = (0.139 + 0.258 + 0.255) * 0.1 / 3
    coulomb_neg = (-0.074 - 0.280 - 0.265) * 0.1 / 3

    u_fric = []
    for qd in qd_rad_s:
      if qd > minimum_velocity:
        val = viscous_pos * qd + coulomb_pos
        # print(f"Positive velocity: {qd:.6f} rad/s, u: {val:.6f} Nm")
        u_fric.append(val)
      elif qd < -minimum_velocity:
        val = viscous_neg * qd + coulomb_neg
        # print(f"Positive velocity: {qd:.6f} rad/s, u: {val:.6f} Nm")
        u_fric.append(val)
      else:
        u_fric.append(slope_near_zero * qd)
    return np.array(u_fric)


  def calculate_IK(self, p):
    """
    Parameters:
    - p: [x, y] desired end-effector position

    Returns:
```

```
        - 1x2 numpy array. Elbow-up solution only. If unreachable, returns [np.nan, np.nan].
        """
        L1, L2 = self.l1, self.l2
        x, y = p
        r2 = x**2 + y**2

        # Check reachability
        max_reach = (L1 + L2)**2
        min_reach = (L1 - L2)**2
        if r2 > max_reach or r2 < min_reach:
            # return np.array([np.nan, np.nan],
            #                 [np.nan, np.nan]])
            return np.array([np.nan, np.nan])

        # Compute cos(q2) and clamp due to numerical error
        c2 = (r2 - L1**2 - L2**2) / (2 * L1 * L2)
        c2 = np.clip(c2, -1.0, 1.0)

        # Two possible q2 values
        q2a = np.arccos(c2)
        q2b = -q2a

        # Corresponding q1 values
        q1a = np.arctan2(y, x) - np.arctan2(L2 * np.sin(q2a), L1 + L2 * np.cos(q2a))
        q1b = np.arctan2(y, x) - np.arctan2(L2 * np.sin(q2b), L1 + L2 * np.cos(q2b))

        # return np.array([[q1a, q2a], [q1b, q2b]])
        return np.array([q1a + pi/2, q2a])  # Only need elbow-up.



import numpy as np

def function_G(q, g, m1, m2,l1x,l1y,l2x,l2y,l1,l2):

    q1 = q[0]
    q2 = q[1]
    q12 = q1 + q2
    sin12 = np.sin(q12)
    cos12 = np.cos(q12)
    sin1 = np.sin(q1)
    cos1 = np.cos(q1)


    G = np.zeros((2, 1))

    G[0, 0] = g*(m2*l2y*cos12 + m2*l2x*sin12 + m1*l1y*cos1 + m2*l2*sin1 + m1*l1x*sin1)
    G[1, 0] = m2*g*(l2y*cos12 + l2x*sin12)

    return G

def function_J14(q, L1, L2):
    q1, q2 = q[0], q[1]

    q1_shift = q1 - np.pi / 2
    sin_q1s = np.sin(q1_shift)
    cos_q1s = np.cos(q1_shift)
    sin_q2 = np.sin(q2)
    cos_q2 = np.cos(q2)

    common = L2 * (cos_q2 * sin_q1s + cos_q1s * sin_q2)

    J14 = np.zeros((2, 3))
    J14[0, 1] = -common - L1 * sin_q1s
    J14[0, 2] = -common
    J14[1, 1] = L2 * (cos_q2 * cos_q1s - sin_q2 * sin_q1s) + L1 * cos_q1s
    J14[1, 2] = L2 * (cos_q2 * cos_q1s - sin_q2 * sin_q1s)

    return J14

def function_p14(q, L):

    q1, q2 = q[0], q[1]
    L1, L2 = L[0], L[1]
```

```
    q1_shift = q1 - np.pi / 2
    sin_q1s = np.sin(q1_shift)
    cos_q1s = np.cos(q1_shift)
    sin_q2 = np.sin(q2)
    cos_q2 = np.cos(q2)

    p14 = np.zeros((2, 1))
    p14[0, 0] = L2 * (cos_q2 * cos_q1s - sin_q2 * sin_q1s) + L1 * cos_q1s
    p14[1, 0] = L2 * (cos_q2 * sin_q1s + sin_q2 * cos_q1s) + L1 * sin_q1s

    return p14


from leg_manager import legManager
from math import pi
from controllers.conrad_controller import *
import matplotlib.pyplot as plt
import numpy as np
import sys
import time
import msvcrt  # For Windows keyboard input
import os
from datetime import datetime
import logging

# Logging setup
logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(levelname)s] %(message)s')
logger = logging.getLogger(__name__)

# User-modifiable parameters
bear_port = 'COM3'
sensor_port = 'COM20'
land_delay_s = 2    # seconds to wait after landing before returning to IDLE
max_duration_s = 5  # cap recording time
Ts = 0.001          # add delay between loops
num_samples = 2000

# Desired positions in operation space
P_DES = {
    'IDLE': [0, -0.25],
    'AERIAL': [0 -0.2625],
    'STANCE': [-0.005 -0.275],
}

# Gains (Aerial and Stance phases in task space, Jump phase in joint space)
KP = {
    'IDLE': np.diag([3, 4]),
    'AERIAL': np.diag(250 * [1, 2]),
    'JUMP': np.diag(1.5 * [1, 0.75]),
    'STANCE': np.diag(150 * [1, 1.25]),
    }
KD = {
    'IDLE': np.diag([0.2, 0.2]),
    'AERIAL': np.diag(100 * [1, 0.875]),
    'JUMP': np.diag(0.0875 * [1, 1]),
    'STANCE': np.diag(15* [1, 1]),
}

# Initialize bear manager and desired controller
leg  = legManager(bear_port=bear_port, sensor_port=sensor_port)
controller = ConradController()  # Leave blank if using the default values

# Prompt user to start control loop
input("Press Enter to start control loop. Press Esc anytime to exit.")
leg.enable_torque(1)

# # For data collection
# pos_desired = []
# pos_measured = []
# start_time = time.time()
current_state = 'IDLE'  # Start in IDLE phase
q_des = controller.calculate_IK(P_DES[current_state])
input(f"Current state: {current_state}. Press Enter to continue to AERIAL state.")
# Finite state machine
current_state = 'AERIAL'  # Start in AERIAL phase
```

```
elapsed_time = None  # Initialize elapsed time for LAND state
leg.jump_flag = False  # Initialize jump flag
leg.previous_contact = False  # Initialize previous contact state
leg.stance_time = 0.75
# q_des = controller.calculate_IK(P_DES[current_state])

while True:
  # Get feedback
  iq, q_rad, qd_rad_s = leg.get_feedback()

  # # save data for plotting
  # current_time = time.time() - start_time
  # pos_desired.append([current_time, q_des[0], q_des[1]])
  # pos_measured.append([current_time, q_rad[0], q_rad[1]])
  # if len(pos_desired) > num_samples:
  #     print("Max duration reached, exiting data collection.")
  #     break

  # Get foot sensor state, see if on the ground or in the air
  # Determine the phase based on foot sensor state (HIGH means in the air, LOW means on the ground)
  if leg.get_foot_sensor_state():
    current_state = 'AERIAL'
    leg.previous_contact = False  # Update previous phase
  else:
    # Check a flag to determine if phase should be JUMP or STAND
    if hasattr(leg, 'stand_flag') and leg.jump_flag:
      current_state = 'JUMP'
      leg.previous_contact = True  # Update previous phase
    else:
      current_state = 'STAND'
      leg.previous_contact = True  # Update previous phase
  if leg.get_foot_sensor_state() == False and leg.previous_contact == False:
    # If the foot sensor was triggered with contact, and the previous phase was aerial, set contact time
    contact_time = time.time()  # Record the time of contact
    logger.info(f"Foot sensor triggered. Contact time recorded at {contact_time:.2f} seconds.")

  # State transitions
  if msvcrt.kbhit():
    key = msvcrt.getch()
    if key == b'\r':  # Enter key
      if current_state == 'IDLE':
        current_state = 'AERIAL'
        logger.info("State changed to AERIAL")
      elif leg.jump_flag == True and current_state == 'AERIAL':  # Ensures that we don't switch to STAND if we are in JUMP phase
        leg.jump_flag = False  # Set jump flag to true
      else:
        leg.jump_flag = True  # Set jump flag to true
    elif key == b'\x1b':  # Esc key
      if current_state == 'STAND':
        logger.info("Esc pressed. Exiting control loop...")
        leg.enable_torque(0)
      else:
        logger.info("Wait until stance to exit control loop next time . :)")
        leg.enable_torque(0)
      break
  kp, kd = KP[current_state], KD[current_state]

  # Compute control action
  u = controller.calc_control_torque(q_des, q_rad, qd_rad_s, kp, kd,current_state,leg)
  # print(f"calculated control torque: {u}")
  leg.move_motors(u)

  # Delay based on sampling rate
  time.sleep(Ts)

# final_time = time.time() - start_time
# print(f"True sampling rate: {len(pos_desired) / final_time:.2f} Hz")

# print(len(pos_desired), len(pos_measured))
# pos_desired = np.array(pos_desired)
# pos_measured = np.array(pos_measured)

# fig, axs = plt.subplots(1, 2, figsize=(14, 6))

# # Joint 1
```

```
# axs[0].plot(pos_desired[:, 0], pos_desired[:, 1], label='Desired Position', linestyle='--')
# axs[0].plot(pos_measured[:, 0], pos_measured[:, 1], label='Measured Position')
# # Add desired position ±0.01 as dashed lines
# axs[0].plot(pos_desired[:, 0], pos_desired[:, 1] + 0.01, 'k--', alpha=0.5, linewidth=0.8, label='Desired +0.01 rad')
# axs[0].plot(pos_desired[:, 0], pos_desired[:, 1] - 0.01, 'k--', alpha=0.5, linewidth=0.8, label='Desired -0.01 rad')
# axs[0].set_xlabel('Time (s)')
# axs[0].set_ylabel('Joint 1 Position (rad)')
# axs[0].set_title('Joint 1: Desired vs Measured')
# axs[0].legend()
# axs[0].grid(True)

# # Joint 2
# axs[1].plot(pos_desired[:, 0], pos_desired[:, 2], label='Desired Position', linestyle='--')
# axs[1].plot(pos_measured[:, 0], pos_measured[:, 2], label='Measured Position')
# # Add desired position ±0.01 as dashed lines
# axs[1].plot(pos_desired[:, 0], pos_desired[:, 2] + 0.01, 'k--', alpha=0.5, linewidth=0.5, label='Desired +0.01 rad')
# axs[1].plot(pos_desired[:, 0], pos_desired[:, 2] - 0.01, 'k--', alpha=0.5, linewidth=0.5, label='Desired -0.01 rad')
# axs[1].set_xlabel('Time (s)')
# axs[1].set_ylabel('Joint 2 Position (rad)')
# axs[1].set_title('Joint 2: Desired vs Measured')
# axs[1].legend()
# axs[1].grid(True)

# plt.tight_layout()
# plt.show()

# # Save logic
# csv_base = "pd_w_gravity"
# date_str = datetime.now().strftime("%Y%m%d")
# csv_base = f"pd_w_gravity_{date_str}"
# data = np.hstack([pos_desired, pos_measured[:, 1:]])
# csv_dir = "experiments/pd_w_gravity"
# os.makedirs(csv_dir, exist_ok=True)
# i = 0
# while True:
#     csv_path = os.path.join(csv_dir, f"{csv_base}_{i}.csv")
#     plot_path = os.path.join(csv_dir, f"{csv_base}_{i}.png")
#     if not os.path.exists(csv_path) and not os.path.exists(plot_path):
#         break
#     i += 1
# np.savetxt(csv_path, data, delimiter=",", header="time, q1_des, q2_des, q1_actual, q2_actual", comments='')
# fig.savefig(plot_path, dpi=300)
# print(f"Saved data to: {csv_path}")
# print(f"Saved plot to: {plot_path}")

import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import sys, os
from datetime import datetime
from scipy.integrate import cumulative_trapezoid
import numpy as np

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'controllers')))
from pd_grav_comp_controller import pdGravCompController

'''
Expected data format in CSV:
First row are headers for the columns:
Timestamp, foot_state, q1_des, q1_meas, q1d_des, q2d_meas, u1_des, u1_meas, q2_des, q2_meas, q2d_des, q2d_meas, u2_des, u2_meas
'''

def calc_z_velocity(q1, q1d, q2, q2d):
    l1, l2 = 0.125, 0.215
    sin_q1 = np.sin(q1)
    cos_q1 = np.cos(q1)
    sin_q1q2 = np.sin(q1 + q2)
    cos_q1q2 = np.cos(q1 + q2)

    # Jacobian entries
    J11 = -l1 * sin_q1 - l2 * sin_q1q2
    J12 = -l2 * sin_q1q2
    J21 =  l1 * cos_q1 + l2 * cos_q1q2
    J22 =  l2 * cos_q1q2

    # Compute xdot and ydot
```

```
    xdot = J11 * q1d + J12 * q2d
    ydot = J21 * q1d + J22 * q2d

    return np.stack([xdot, ydot], axis=1)  # shape (N, 2)

def plot_shaded_region(ax):
    for start, end in zip(starts, ends):

        ax.axvspan(start, end, color='tab:purple', alpha=0.2)

import numpy as np

def get_torque_bounds(omega):
    # Constants
    kv = 9 * 2 * 6      # deg/s/V
    no_load_speed = kv * 24      # max voltage = 24 V
    stall_torque = 1.16 * 20     # kt = 1.16 Nm/A, max current = 20 A

    # Constraints: A @ [omega; tau] <= B
    A = np.array([
        [0, 1],
        [0, -1],
        [stall_torque / no_load_speed, 1],
        [-stall_torque / no_load_speed, -1]
    ])
    B = np.array([23.2, 23.2, stall_torque, stall_torque])

    b = B - A[:, 0] * omega
    a2 = A[:, 1]

    upper = b[a2 > 0] / a2[a2 > 0]
    lower = b[a2 < 0] / a2[a2 < 0]

    if upper.size == 0 or lower.size == 0 or np.max(lower) > np.min(upper):
        return float('nan'), float('nan')

    return np.max(lower), np.min(upper)

# Load data from CSV file and trim if needed
filepath = r"C:\Users\c_mku\Documents\MAE263C\MAE-263C-Project\Leg_Test\experiments\pd_w_gravity_jump\matlab_sim_03.csv"
front_trim = 0
back_trim = 200
if not os.path.isfile(filepath):
    raise FileNotFoundError(f"File not found: {filepath}")
# recorded_data = np.loadtxt(filepath, delimiter=',', skiprows=1)
recorded_data = np.genfromtxt(filepath, delimiter=",", dtype=np.float64, filling_values=np.nan)
recorded_data = recorded_data[~np.isnan(recorded_data).any(axis=1)]
# print(recorded_data)
back_trim = len(recorded_data) - back_trim
recorded_data = recorded_data[front_trim:back_trim]

# Process foot sensor to graph shaded region
time = recorded_data[:, 0]
contact_raw = recorded_data[:, 1]
contact_clean = (contact_raw > 0.5).astype(int)  # Add debouncing here later if needed
contact_diff = np.diff(contact_clean, prepend=0)
starts = time[contact_diff == 1]
ends = time[contact_diff == -1]
starts = np.append(starts, 4.16)
ends = np.append(ends, time[-1])

# Assign columns to np arrays
q1_des = recorded_data[:, 2]
q1_meas = recorded_data[:, 3]
q1d_des = recorded_data[:, 4]
q1d_meas = recorded_data[:, 5]
u1_des = recorded_data[:, 6]
u1_meas = recorded_data[:, 7]
q2_des = recorded_data[:, 8]
q2_meas = recorded_data[:, 9]
q2d_des = recorded_data[:, 10]
q2d_meas = recorded_data[:, 11]
u2_des = recorded_data[:, 12]
u2_meas = recorded_data[:, 13]
```

```
Z_height = recorded_data[:, 14]

x_des = recorded_data[:, 20]
xd_des = np.gradient(x_des, time)
x_meas = recorded_data[:, 21]
xd_meas = np.gradient(x_meas, time)
z_des = recorded_data[:, 22]
zd_des = np.gradient(z_des, time)
z_meas = recorded_data[:, 23]
zd_meas = np.gradient(z_meas, time)

hip_tau_min, hip_tau_max = [], []
knee_tau_min, knee_tau_max = [], []
for w1, w2 in zip(np.rad2deg(q1d_meas), np.rad2deg(q2d_meas)):
    tmin1, tmax1 = get_torque_bounds(w1)
    tmin2, tmax2 = get_torque_bounds(w2)
    hip_tau_min.append(tmin1)
    hip_tau_max.append(tmax1)
    knee_tau_min.append(tmin2)
    knee_tau_max.append(tmax2)
hip_tau_min, hip_tau_max = np.array(hip_tau_min), np.array(hip_tau_max)
knee_tau_min, knee_tau_max = np.array(knee_tau_min), np.array(knee_tau_max)

# Calculate EE velocity using jacobian
jac_vel = calc_z_velocity(q1_meas, q1d_meas, q2_meas, q2d_meas)

# Converts joint angles to operation space with FK
leg = pdGravCompController()
p_des, p_meas = [], []
for data_point in recorded_data:
    # print(data_point)
    p_des.append(leg.calculate_FK([data_point[2], data_point[8]]))
    p_meas.append(leg.calculate_FK([data_point[3], data_point[9]]))
p_des = np.array(p_des)
p_meas = np.array(p_meas)
# print(p_des)

######### PLOTTING #########
# 8 subplots for maximum readability
plt.rc('font', family='serif')
fig, axs = plt.subplots(4, 2, figsize=(10, 10))
fig.subplots_adjust(wspace=0.35)

# Plot shaded region as aerial phase
for i in range(4):
    for j in range(2):
        if i == 3 and j == 0:
            continue  # Skip axs[3][0] only
        plot_shaded_region(axs[i][j])

# # X
# axs[0][0].plot(time, x_des, label='Desired', color='tab:blue', linestyle='--', alpha=0.5)
# axs[0][0].plot(time, x_meas, label='Measured', color='tab:blue')
# axs[0][0].set_xlabel('Time (s)')
# axs[0][0].set_ylabel('End Effector X Position (m)')
# axs[0][0].legend(fontsize=9)

# # X vel
# axs[1][0].plot(time, xd_des, label='Desired', color='tab:blue', linestyle='--', alpha=0.5)
# axs[1][0].plot(time, xd_meas, label='Measured', color='tab:blue')
# axs[1][0].set_xlabel('Time (s)')
# axs[1][0].set_ylabel('End Effector X Velocity (m/s)')

# Joint 1 Angle
axs[0][0].plot(time, np.rad2deg(q1_des), label='Desired', color='tab:blue', linestyle='--', alpha=0.5)
axs[0][0].plot(time, np.rad2deg(q1_meas), label='Measured', color='tab:blue')
axs[0][0].set_xlabel('Time (s)')
axs[0][0].set_ylabel('Hip Joint Angle (deg)')
axs[0][0].legend(fontsize=9)

# Joint 1 Velocity
axs[1][0].plot(time, np.rad2deg(q1d_des), label='Desired', color='tab:blue', linestyle='--', alpha=0.5)
axs[1][0].plot(time, np.rad2deg(q1d_meas), label='Measured', color='tab:blue')
axs[1][0].set_xlabel('Time (s)')
axs[1][0].set_ylabel('Hip Joint Velocity (deg/s)')
```

```
# Joint 1 Torque
axs[2][0].plot(time, u1_des, label='Desired', color='tab:red', linestyle='--', alpha=0.5)
axs[2][0].plot(time, u1_meas, label='Measured', color='tab:red')
axs[2][0].plot(time, hip_tau_min, 'k--', linewidth=1)
axs[2][0].plot(time, hip_tau_max, 'k--', linewidth=1)
axs[2][0].set_xlabel('Time (s)')
axs[2][0].set_ylabel('Hip Joint Torque (Nm)')
axs[2][0].legend(fontsize=9)

# Joint 2 Angle
axs[0][1].plot(time, np.rad2deg(q2_des), label='Desired', color='tab:orange', linestyle='--', alpha=0.5)
axs[0][1].plot(time, np.rad2deg(q2_meas), label='Measured', color='tab:orange')
axs[0][1].set_xlabel('Time (s)')
axs[0][1].set_ylabel('Knee Joint Angle (deg)')
axs[0][1].legend(fontsize=9)

# Joint 2 Velocity
axs[1][1].plot(time, np.rad2deg(q2d_des), label='Desired', color='tab:orange', linestyle='--', alpha=0.5)
axs[1][1].plot(time, np.rad2deg(q2d_meas), label='Measured', color='tab:orange')
axs[1][1].set_xlabel('Time (s)')
axs[1][1].set_ylabel('Knee Joint Velocity (deg/s)')

# Joint 2 Torque
axs[2][1].plot(time, u2_des, label='Desired', color='tab:red', linestyle='--', alpha=0.5)
axs[2][1].plot(time, u2_meas, label='Measured', color='tab:red')
axs[2][1].plot(time, knee_tau_min, 'k--', linewidth=1)
axs[2][1].plot(time, knee_tau_max, 'k--', linewidth=1)
axs[2][1].set_xlabel('Time (s)')
axs[2][1].set_ylabel('Knee Joint Torque (Nm)')

# Operation space desired vs actual
axs[3][0].plot(p_des[:, 0], p_des[:, 1], label='Desired', color='tab:green', linestyle='--', alpha=0.5)
axs[3][0].plot(p_meas[:, 0], p_meas[:, 1], label='Measured', color='tab:green')
axs[3][0].set_xlabel('X Position (m)')
axs[3][0].set_ylabel('Z Position (m)')
axs[3][0].yaxis.set_major_formatter(ticker.FormatStrFormatter('%.2f'))
axs[3][0].axis('equal')
axs[3][0].legend(fontsize=9)

# # IMU reading
# axs[3][1].plot(time, imu_g, label='IMU', color='tab:brown')
# axs[3][1].set_xlabel('Time (s)')
# axs[3][1].set_ylabel('Acceleration in Z (g)')

# # Integrated IMU reading
# # axs[3][1].plot(time, int_vel_z, label='From IMU', color='tab:brown')
# axs[3][1].plot(time, jac_vel[:, 1], label='From Jacobian', color='tab:olive')
# # axs[3][1].plot(time, np.gradient(p_meas[:, 1], time), label='Velocity by Diff p_meas', color='tab:cyan')
# axs[3][1].set_xlabel('Time (s)')
# axs[3][1].set_ylabel('Z velocity (m/s)')
# axs[3][1].legend(fontsize=9)
in_contact = np.zeros_like(time, dtype=bool)
axs[3][1].plot(time, Z_height, label='Measured', color='tab:brown')
for start, end in zip(starts, ends):
    in_contact = (time >= start) & (time <= end)
    axs[3][1].plot(time[in_contact], np.full(np.sum(in_contact), 0.275), '--', label = 'Desired',color='saddlebrown', linewidth=1)
axs[3][1].set_xlabel('Time (s)')
axs[3][1].set_ylabel('Absolute Z height')
# axs[3][1].legend(fontsize=9)

# Add subplot labels A) to H)
labels = ['A)', 'B)', 'C)', 'D)', 'E)', 'F)', 'G)', 'H)']
for idx, ax in enumerate(axs.flat):
    ax.text(-0.15, -0.1, labels[idx], transform=ax.transAxes, fontsize=10, va='top', ha='left', fontweight='bold')

plt.tight_layout()
plt.savefig(filepath.replace('.csv', '.png'), dpi=300)
plt.show()
```